CSC8507/08 游戏开发

文档信息

表格

	修改者	修改时间	修改内容
1	喻健然	2026/01/28	创建文档总体框架，增加表格示例
2	董一均	2026/01/28	1.4 敌人与 AI 行为
3	褚姝良	2026/02/19	1.3玩家角色与操作
4			

4 条记录

1. 需求说明 （功能性）

1.1 核心游戏主循环

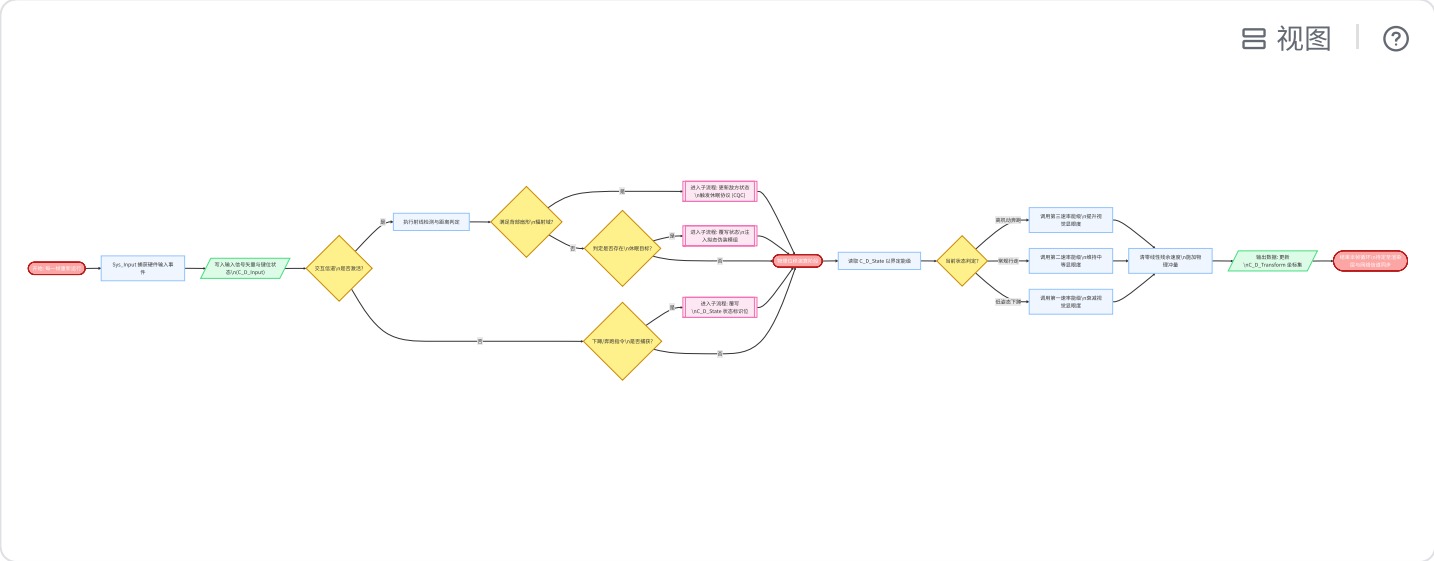
1.2 关卡与世界结构

我们将会重构空间坐标到类似unity的格式

1.3 玩家角色与操作

基于 NCLGL 引擎与 ECS 架构实现，主要有玩家实体的构建、跨平台输入响应、基于物理的移动控制以及非暴力潜行交互。逻辑处理主要集中在 Sys_Input、Sys_Movement 和 Sys_Gameplay 三个系统中。

ECS 纯逻辑行为树：



1.3.1 玩家实体的界定与组件装配规范 (Definition and Component Assembly Protocols of the Player Entity)

玩家实体（`Entity_Player`）实例化须遵循数据驱动原则，严格配置以下组件以实现逻辑解耦：

1. **身份标签**：挂载 `C_T_Player`。该标签不涉运算逻辑，仅作摄像机目标锚定及敌方 AI 索敌矩阵的身份鉴别标识。
2. **碰撞层级**：设为 `COLLISION_PLAYER`。仅与场景墙体、敌方及触发器干涉，严禁与抛射物重叠，以防引发异常物理阻滞。
3. **数据组件**：集成 `C_D_Transform`（空间坐标与旋转矩阵）、`C_D_Input`（当前帧输入缓存）、`C_D_State`（状态机标识位）及 `C_D_Health` / `C_D_Inventory`（生命与物料库存管理）。

1.3.2 跨平台输入信号的解析与映射规范 (Resolution and Mapping Protocols of Cross-Platform Input Signals)

`Sys_Input` 系统每帧采集底层硬件输入，转化为统一的 `C_D_Input` 标准格式，隔离多平台底层差异：

1. **PC 终端映射**：W/A/S/D 控制二维四向矢量；E 键触发场景交互；鼠标左键施放道具；Ctrl/C 切换下蹲状态；Shift 维持高速奔跑。
2. **主机终端映射**：左摇杆控制偏转向量；□/× 键触发场景交互；L2/R2 执行动作确认与道具施放；L1/R1 轮询切换物料槽位。

1.3.3 基于物理引擎的位移演算机制（Physics-Based Displacement Calculation Mechanisms）

空间位移由 Jolt Physics 引擎在 `Sys_Movement` 系统的固定帧（FixedUpdate）内集中解算，施加以下物理约束：

1. **视椎体相对位移**：提取摄像机朝向矢量（Forward/Right），将输入坐标系转换为绝对位移向量，确保控制方向与视觉焦点共线。
2. **零惯性阻尼控制**：在位移启停帧强制清空刚体线性速度（Velocity），实现瞬时制动，保障潜行规避时的位置精度。
3. **轴向刚性锁定**：锁定刚体绕 X/Z 轴的旋转自由度，仅允许绕 Y 轴（垂直法线）偏转，消除场景物理碰撞引发的非预期倾倒。

1.3.4 状态机及速度与显眼度耦合系统（Entity State Machine and Coupled Velocity-Visibility System）

实体的移动标量与敌方视觉侦测矩阵中的显眼度（Visibility）受控于互斥状态机。其中奔跑状态具最高优先级，可强制中断下蹲：

1. **常规行走 (Walk)**：缺省状态。速率限为第二能级（匀速 1.0），视觉显眼度维持中等，属主要探索模式。
2. **低姿态下蹲 (Crouch)**：触发式模式。速率衰减至第一能级（慢速 0.5），视觉显眼度同步下调，专用于抑制高危区域中敌方警戒阈值的累计速度。
3. **高机动奔跑 (Run)**：按压式模式。速率提升至第三能级（全速 1.5），显眼度同步攀升至中高档，主要用于脱离追踪或穿越确认安全域。

1.3.5 核心交互与非暴力渗透机制（Core Interactive Behaviors and Non-Violent Infiltration Mechanisms）

交互逻辑由 `Sys_Gameplay` 结合射线检测（Raycast）与敌方 AI 状态统一判定，严格执行非暴力准则：

1. **近距离控制协议 (CQC)**：执行实体需位于敌对目标背部特定扇形辐射域内，且目标未处于“追捕”状态。触发后禁止渲染暴力特效，通过投射骇入与“内存溢出”代码特效，迫使目标强制休眠。
2. **拟态伪装机制 (Mimicry)**：针对已休眠目标再次交互触发。系统直接覆写执行实体的网格与材质资产，于 `C_D_State` 注入拟态标志位。该标志位可令同阵营 AI 的视觉侦测执行旁路（Bypass）操作，直至发起攻击注销。

1.4 敌人与 AI 行为

ECS 纯逻辑行为树：

1.5 交互机制

1.6 UI 与游戏流程

1.7 联机与合作模式

1.8 服务器逻辑

为了防止逻辑冲突，关键逻辑放在服务器进行判断：

- **玩家角色 (Player)**：
  - **位置/旋转**：采用 **客户端授权 (Client Authority)**。客机直接控制自己的角色移动并上传坐标，主机接受并广播。
  - **状态/数值**（如血量、道具）：采用 **服务器授权 (Server Authority)**。必须由主机判定后下发。
- **AI 敌人**：
  - **完全服务器授权**。AI 逻辑仅在主机运行，客机仅接收 AI 的位置和动画状态进行“傀儡化”渲染。

- **场景机关**：
  - **服务器授权**。门、机关等的状态由主机管理。

1.9 游戏物理

因为我们需要处理多线程优化也就是同时处理大量刚体，并且需要跨平台开发表现，所以我们选取了第三方物理引擎Jolt。

ECS 的组件（如 C_D_Transform）只存纯数据和 Jolt 的 BodyID，由 Sys_Physics 在 OnFixedUpdate 阶段负责将 ECS 数据推给 Jolt，执行 Jolt 步进，然后再将 Jolt 的结果（动态物体位置、碰撞事件）拉回 ECS。

游戏采用第三人称 2.5D 视角，场景中存在一定高度差起伏。物理世界保持纯 3D 碰撞运算，我们保留高度位移，以适应地形变化，且锁定多余的旋转和平面外的位移。

1.10 可选拓展与可能的加分要素

1.11 功能性需求表

表格

	编号	名称	所属阶段	类型	优先级	简要说明
1	FR-01	单机核心游戏...	Part A	核心玩法	必须	从主菜单进入单机关卡，
2	FR-02					
3	FR-03					
4	FR-04					
5	FR-04					

5 条记录

2. 需求说明（非功能性 - 架构）

2.1 总体设计原则

2.2 模块划分与职责

2.3 依赖与耦合策略

2.4 可维护性与可读性

2.5 可测试性与可调试性

2.6 性能与资源约束

2.7 可扩展性与重用

2.8 假设与限制

2.9 非功能性需求表

表格

	编号	名称	所属阶段	类型	优先级	简要说明	
1	NFR-01						
2	NFR-02						
3	NFR-03						
4	NFR-04						
5	NFR-05						
6	NFR-06						
7	NFR-07						
8	NFR-08						
9	NFR-09						
10	NFR-10						
11	NFR-11	务实代码规范	全局	代码规范	最高	每个 .h 与 .cpp 文件必须...	所
12	NFR-12						

12 条记录

3. 核心设计规范（fundamental design regulation）

3.1 设计目的（purpose）

介绍如何将 **数据（Component）** 和 **逻辑（System）** 彻底分离

3.2 全局命名与代码风格规范（Global naming and code format）

统一命名，别乱写名称

3.2.1 工程目录结构与文件组织规范(Project catalog structure & file Organization rules)

首先是代码部分：

Code:

项目文件结构应保持清晰、扁平，遵循“代码与资产分离”、“核心与业务分离”的原则。工程根目录严禁存放临时文件（如 .obj, .pdb），所有构建中间产物应由 CMake 或 IDE 自动重定向至 Build/ 目录。

1. 根目录结构规范：(example)

目录名称	说明
Assets/	存放所有游戏运行时资产。内部需按资产类型分子目录（详见下文）。
Source/	存放所有 C++ 源代码文件 (.h, .cpp)。
External/	存放第三方库（如 ImGui, nlohmann_json 等），严禁修改库源码。
Build/	编译输出目录，应在 .gitignore 中忽略。
Docs/	存放策划文档、技术文档及会议记录。

2. Assets（资产）目录详解：（Assets：）

Assets 目录下不应包含源文件，仅包含游戏引擎可读取的资源。（Assets should not contain any source code, this part only contains the loadable files for the engine）

子目录路径	内容说明	命名示例
Assets/Prefabs/	核心数据源。存放预制体 JSON 文件，定义每种实体由哪些组件组成及其默认数值。所有实体的"蓝图"都在这里。	Prefab_Player_Main.json, Prefab_Enemy_Orc.json
Assets/Levels/	存放关卡数据文件（JSON），引用 Prefab ID 来描述场景中的实体布局	Level_Tutorial.json
Assets/Dialogs/	存放对话文件（JSON），对应剧情文本与选项逻辑	Dialog_Details.json
Assets/Configs/	存放游戏数值配置、AI 行为树配置等全局参数。	EnemyStats.json, BT_Patrol.json
Assets/Textures/	存放所有纹理贴图 (.png, .tga等)。	UI_HealthBar.png
Assets/Models/	存放模型文件 (.obj, .fbx, .gltf) 及对应的材质定义。	Char_Hacker.obj
Assets/Audio/	存放音效与背景音乐 (.wav, .mp3)。	BGM_Cyberpunk.wav
Assets/Shaders/	存放着色器代码 (.vert, .frag，.geom)。	Shader_Toon.vert

3. Source（源码）目录详解（Source：）

源码分为 Engine（核心底层）与 Game（游戏逻辑）两大部分。Game 目录严格遵循 ECS 架构进行分层。

Source contains Engine（base）and Game（logic）. Game dictionary follows ECS structure restrictly.

子目录路径	说明	包含文件类型
Source/NCLCore/	存放 NCL 引擎原始源码。除必要的 Bug 修复外，原则上不在此目录添加新功能，将其视为只读库	所有NCLGL引擎的内容
Source/Core/	核心架构层根目录。其中包含 ECS 框架实现及与 OOP 引擎的交互适配接口	(无，作为中间目录)
Source/Core/ECS/	ECS 基础库实现	Registry.h, BaseSystem.h
Source/Core/Bridge/	中间层，负责将 NCL 的 OOP 对象（如 GameObject）与 ECS 组件进行双向绑定的“胶水代码” `Bridge` 文件夹内**不允许保存任何状态（State）**。状态只能在 ECS 的 Component 中。Bridge 只能包含转换函数	NCL_RenderBridge.h, Input_Adapter.cpp
Source/Game/	具体游戏业务逻辑根目录。	(无，作为中间目录)
Source/Game/Components/	存放 POD struct 定义，配合 REFLECT_STRUCT 宏注册元数据，供 PrefabFactory 从 JSON 自动映射数据。严禁包含复杂逻辑。	C_D_Health.h, C_T_Player.h
Source/Game/Systems/	系统实现。包含具体的逻辑处理函数，每帧更新组件数据。	Sys_Movement.cpp, Sys_AI.cpp
Source/Game/Events/	事件定义。用于系统间解耦通信的数据包。	Evt_Collision.h
Source/Game/Prefabs/	**工厂与反射代码**。存放 PrefabFactory C++ 实现（解析 JSON、通过反射实例化 Entity）以及组件反射注册代码。**注意**：JSON 数据文件存放在 Assets/Prefabs/，此目录仅存放 C++ 代码。	PrefabFactory.h, UnitBuilder.cpp
Source/Game/Utils/	游戏层通用的辅助工具或工厂类。	timer.h

3.2.2 实体 (Entity) 变量与调试命名规范(Entity&Debugging)

首先是C++ 变量命名规范：

在代码中持有 EntityID 的局部变量或成员变量，应该采用**entity_[主类型]_[描述1]_[描述2（如编号或者详细分类）]_[描述3（如编号）]**的方式命名

示例：

`entity_Player_Main` (玩家实体)

`entity_Enemy_Orc_01` (敌人实体)

`entity_Item_Key` (道具实体)

然后是在调试时的命名规范：

Debug：

作为 `DebugNameComponent` 组件存储的字符串值，以及在 Log/Console 输出的内容。 为了与System或 Component等引擎模块日志区分，实体日志名称必须使用全大写前缀。

格式：**[全大写前缀]_[主类型]_[描述1]_[描述2（如编号或者详细分类）]_[描述3（如编号）]**

示例： `ENTITY_Player_Main` (玩家实体)

> ⚠ 注意：所有实体在 `PrefabFactory` 中创建时，必须第一时间挂载 `DebugNameComponent` 并赋予符合上述规范的字符串。

3.2.3 组件 (Component) 与 标签 (Tag) 命名规范(Component & Tag)

首先是C++命名规范：

在代码中定义组件或标签的结构体（struct）时，应采用以下分层前缀命名，利用自动补全功能进行快速筛选。

数据组件：包含实际业务数据的结构体，作为逻辑运算的数据源。

格式：**C_D_[功能名称]**

示例：

- C_D_Health (包含 hp, max_hp)
- C_D_Transform (包含 pos, rot等)
- C_D_Input (包含 key_state等)

标签组件：不包含数据（或仅包含极少状态位）的空结构体，仅用于实体筛吗，选与分类。

格式：**C_T_[身份/状态名称]**

示例：

- C_T_Player (玩家标记)
- C_T_Enemy (敌人标记)

    - C_T_Bullet　(子弹标记)

然后是在调试时的命名规范：

用于 Log 输出和组件反射（Reflection）系统的字符串标识。 采用全大写风格，前缀与类型定义保持严格对应。

格式：**[全大写前缀]_[全大写主类型]_[描述1]_[描述2（如详细分类）]**

示例：

C_D_Health

C_T_Player

文件命名规范：

文件命名应与类名严格保持一致（大小写敏感），一个文件仅包含一个组件或标签，如：

Source/Game/Components/C_D_Health.h

3.2.4 系统 (System) 命名规范(System)

系统是 ECS 架构中的逻辑执行者，负责处理组件数据

首先是C++命名规范：

在代码中定义系统类（class）时，必须以 Sys_ 开头，文件命名与类名保持一致

格式：**Sys_[功能名称]**

示例：

- Sys_Movement　(处理位置移动与物理模拟)
- Sys_Rendering　(处理网格绘制与动画)
- Sys_AI_Behavior (处理行为树决策)

然后是核心接口规范

Ports:

所有 System 类必须实现以下核心生命周期函数，以保证引擎主循环的统一调用：

- void OnUpdate(float dt);
  - 说明：每帧调用，dt 为上一帧的间隔时间 (Delta Time)。
- void OnFixedUpdate(float fixedDt);
  - 说明：每帧调用，fixedDt为固定时间，用于物理模拟 (Sys_Movement)、AI决策、网络状态同步。
- void OnAwake(); / void OnDestroy();
  - 说明：系统的初始化与销毁回调。

然后是在调试时的命名规范：

Debugging：

用于性能分析器 (Profiler) 标记或系统运行日志。 采用全大写风格，前缀与类型定义保持严格对应。

格式：**SYS_[功能名称]**

示例：

- 类名：Sys_Movement
- 调试名："SYS_MOVEMENT"
- 用途：[PROFILER] SYS_MOVEMENT took 2.5ms.

3.2.5 事件 (Event) 命名规范(Event)

事件是系统（System）之间解耦通信的载体，通常定义为瞬时的结构体数据。命名应清晰表明“发生了什么事情”。

首先是C++命名规范：

C++ Naming:

在代码中定义事件结构体时，必须采用Evt_ 前缀，文件命名与类名保持一致

格式：**Evt_[主类型/模块]_[具体行为]**

示例：

- Evt_Phys_Collision （物理模块：发生碰撞)
- Evt_Game_LevelStart (游戏流程：关卡开始)
- Evt_Input_Jump （输入模块：跳跃指令)

然后是事件结构体内的成员变量命名规范：

结构体成员变量应保持简洁，变量名仅描述数据的本质含义，**严禁**重复结构体名称作为前缀：

示例：

代码块

```
struct Evt_Phys_Collision {
        EntityID entity_a;   // 清晰
        EntityID entity_b;
        float impact_force;  // 清晰
    };
```

错误示例（请勿模仿）：

代码块

```
struct Evt_Phys_Collision {
        EntityID evt_phy_coll_entity_a; // 冗余，调用时会变成 event.evt_phy_coll_entity_a
    };
```

在系统函数中处理事件时，建议使用 evt 作为前缀来命名事件实例，以增强辨识度。

代码块

```
void OnCollision(Evt_Phys_Collision& evt_collision) {
        // 通过实例名区分，而不是通过成员名区分
        ProcessDamage(evt_collision.entity_a, evt_collision.impact_force);
    }
```

然后是在调试时的命名规范：

Debugging:

用于资源管理器或控制台变量的查询。 采用全大写风格，前缀与类型定义保持严格对应。

格式：**EVT_[功能名称]**

示例：

- 类名：Evt_Phys_Collision
- 调试名："EVT_PHYS_COLLISION"

3.2.6 全局资源 (Resource/Context) 命名规范(Global Resource & Context)

全局资源是指不依附于特定实体、在系统间共享的单例数据结构体

首先是C++命名规范：

C++:

资源结构体应采用 Res_ 前缀，明确其作为“全局数据”的身份。

格式：**Res_[功能名称]**

示例：

- Res_Time　　(包含 dt, time_scale 等时间数据)
- Res_Input　　(包含 mouse_pos, key_states 等输入状态)
- Res_GameSettings (包含 volume, brightness 等设置)
- Res_GlobalState (包含当前警戒度、倒计时等游戏流程数据)

然后是变量命名规范：

在系统 (System) 中访问资源时，建议使用 res_ 作为变量前缀：

示例：

代码块

```
void OnUpdate(float dt) {
      auto& res_input = registry.GetResource<Res_Input>();
      if (res_input.is_jump_pressed) { ... }
  }
```

然后是在调试时的命名规范：

用于资源管理器或控制台变量的查询，采用全大写风格，前缀与类型定义保持严格对应。

格式：**RES_[功能名称]**

示例：

- 类名：Res_Time
- 调试名："RES_TIME"
- 日志例子：[INFO] RES_TIME scale set to 1.0.

3.2.7 预制体 (Prefab) 数据定义命名规范(Prefab Defination Conventions)

预制体（Prefab）是指用于生成特定实体的一组组件配置（蓝图）。为平衡开发效率与运行性能，本项目支持“硬编码工厂”与“数据驱动（JSON）”两种模式

首先是JSON字符串命名规范：

JSON:

在需要构建JSON配置文件/关卡内容/代码内部创建索引时的唯一字符串时，其名称应当采用全大写且保持层级结构

格式：**PREFAB_[主类型]_[子类型]_[名称]**

示例：

- PREFAB_PLAYER_MAIN (玩家主角)
- PREFAB_ENEMY_ORC_GRUNT (兽人步兵)
- PREFAB_ITEM_HEALTH_POTION (回复药水)

然后是C++命名规范：

C++:

用于在工厂方法中，负责实例化实体的静态函数的命名

格式：**Create[主类型][名称](Registry&, Position, ...)**

示例：

- CreatePlayerMain(...)
- CreateEnemyOrcGrunt(...)
- CreateItemHealthPotion(...)

然后是JSON文件的命名规范：
JSON:

如果这个p1、refab是由JSON文件数据驱动的话，其文件名应该与ID逻辑对应，便于加载

格式：**Prefab_[主类型]_[名称].json**

示例：

- Assets/Configs/Prefabs/Prefab_Enemy_Orc.json
- Assets/Configs/Prefabs/Prefab_Item_Potion.json

然后是在调试时的命名规范：

用于资源管理器或控制台变量的查询，采用全大写风格，前缀与类型定义保持严格对应。

格式：**PREFAB_[主类型]_[子类型]_[名称]**

示例：

- ID/Key：PREFAB_ENEMY_ORC
- 调试名："PREFAB_ENEMY_ORC"
- 日志用途：[INFO] Spawning PREFAB_ENEMY_ORC at (10, 5).

网络模块 (Network) 命名规范

网络底层协议相关的结构体或类，应统一使用 `Net_` 作为前缀

格式：**Net_[功能/协议名称]**

示例：

- `Net_Packet_Handshake` (握手数据包)
- `Net_Packet_Transform` (位置同步数据包)
- `Net_Client` / `Net_Server` (底层客户端与服务端管理器)

文件命名规范：

网络协议定义文件应以 `Net_` 开头。

示例：

- `Source/Game/Network/Net_Protocol.h`

枚举 (Enum) 命名规范：

网络相关的枚举类型，其成员应当全大写，并且前缀清晰。

例如 `enum PacketType` 里的成员应当以功能分类作为前缀，如 `SYS_` （系统包）、`SYNC_` （同步包）、`GAME_` （业务包）。

3.2.8 常用宏定义 (Macros) 与调试日志规范（Macros&Logs）

为了避免调试代码污染正式版本性能，并确保控制台输出的可读性，**严禁**直接使用 std::cout 或 printf。所有调试输出与断言必须通过封装好的宏进行调用。

首先是C++命名规范：

C++

所有宏定义必须使用全大写字母 (SCREAMING_SNAKE_CASE)，并且必须增加项目前缀 GAME_ 以避免与第三方库冲突。

格式：**[前缀]_[功能]_[描述]**

示例：

- GAME_LOG_INFO(...)
- GAME_ASSERT(condition, msg)
- GAME_PROFILE_FUNCTION()

然后是各个宏的类型：

核心调试日志宏：

日志宏应仅在 Debug 模式下生效（通过 #ifdef _DEBUG 控制），在 Release 模式下应被编译为空指令

示例：

宏名称	用途	格式要求
LOG_INFO(...)	普通信息流，用于流程追踪	[INFO] [Frame:102] Game started.
LOG_WARN(...)	潜在问题，但不影响运行	[WARN] [Frame:105] Texture 'sky.png' missing, using default.
LOG_ERROR(...)	严重错误，逻辑中断	[ERR] [Frame:200] System 'Sys_Physics' crashed!

输出格式规范：

日志必须包含时间戳或帧数，建议采用以下结构：

[LEVEL] [Time/Frame] [Category/DebugName] Message

示例：[INFO] [12.5s] ENTITY_Player_Main spawned at (0,0).

断言与安全宏

用于在开发阶段捕捉逻辑错误。当条件为 false 时，应暂停程序并打印堆栈信息。

示例：

- GAME_ASSERT(condition, message)
  - 比如：GAME_ASSERT(health > 0, "Player health cannot be negative!");
  - 行为：Debug 模式下中断程序 (DebugBreak)；Release 模式下忽略或仅记录错误。

ECS 注册与工具宏

用于简化 ECS 组件注册或反射机制的辅助宏，比如REGISTER_COMPONENT(Type)

  - 用途：利用宏的字符串化功能 (#Type) 自动注册组件的反射信息。
  - 示例：REGISTER_COMPONENT(Comp_Data_Health);

> ⚠ 严禁使用无意义的调试输出，如 `cout << "111" << endl;`。
>
> 严禁在宏内编写带有副作用（Side Effects）的代码。
>
> 错误示例：**GAME_ASSERT(FunctionThatChangesState(), "Check failed");**
>
> 后果：Release 模式下该函数会被剔除，导致游戏逻辑改变。

3.3 ECS 架构核心机制设计约定(ECS Structure Core Mechanism Design Regulation)

3.3.1 注册表 (Registry) 生命周期与管理约定（Registry & LifeCycle）

1. **简介**

   注册表（Registry）是整个ECS架构的心脏，在ECS架构中，数据和逻辑是分离的，而regisrty的作用就是作为一个集中式的数据库——

   - 它负责**分配**和**回收** Entity ID（身份证号）。

- 它负责**存储**所有的 Component 数据（把所有人的 HP 存在一起，把所有人的 Position 存在一起）。
- 它提供**查询接口**（Query），让 System 能快速找到它需要的数据（比如："给我所有同时拥有'位置'和'速度'的实体"）。

事实上，你们可以把它想象成一个超大型的数据库即可

2. **约定**
   a. **单例数据源约定**
      - 在任一时刻（单人或多人模式的 Host 端），**Registry 必须是唯一的**。
      - 严禁 System 内部私自持有实体数据的副本。所有系统必须通过 `registry.view<T>()` 或 `registry.get<T>(entityID)` 实时获取数据，确保逻辑处理的是最新状态。
   b. **内存管理模型**
      - **列式存储**：Registry 内部必须以**组件类型**为维度管理内存。同一类型的组件数据（如 `C_Transform`）在物理内存中必须是**连续存放**的（Contiguous Memory），以最大化 CPU 缓存命中率（Cache Coherence）。
      - **预分配策略**：在 `OnAwake()` 阶段，Registry 应根据预估的关卡规模（如 Max Entities = 10,000），预先 `reserve` 组件池的大小，杜绝游戏运行时的动态扩容（Reallocation）导致的掉帧。
   c. **ID 生命周期管理**
   - **实体 ID 结构**：实体 ID 应为一个 32位或 64位整数，需包含 **Index (索引)** 和 **Version (版本号)** 两部分。
     - *目的*：防止"悬空指针"问题。当 Entity 10 (v1) 被销毁后，该位置被新实体 Entity 10 (v2) 占据，持有旧 ID 的系统若尝试访问，应因版本号不匹配而拒绝访问。

   - **延迟销毁**：也就是我在白板上写的 `Frame End: Clear Events` 那个阶段。
     - System 不应直接 `delete` 实体，而应调用 `registry.destroy(entity)` 打上销毁标记。
       - 真正的内存移除操作统一在 **帧末**执行，防止在一个 System 遍历过程中破坏了另一个 System 的迭代器。
   d. **关卡重置约定**
      - Registry 必须提供 `Clear()` 接口。
      - 切换场景时，`Clear()` 仅重置 Entity 计数器和组件的有效数量（Size），**保留内存容量**。这确保了玩家重试关卡（Retry）时的加载速度是极快的。

3.3.2 场景 (Scene) 生命周期与切换机制（Scene LifeCycle & Switch）

1. 简介

   场景是“**当前生效的系统**”的集合——或者说是一个超级大的舞台，但只有指定的系统能够上台表演

   场景的核心用途有：

   - **状态隔离**：主菜单（MainMenu）不需要物理系统，不需要 AI 系统；而游戏关卡（Gameplay）需要。场景负责定义“当前生效的系统集合”。
   - **资源管理**：场景定义了“这一关需要通过Registry加载哪些 JSON，哪些纹理，哪些音频”。
   - **流程控制**：它负责告诉引擎：“现在是暂停状态”、“现在是游戏进行中”还是“现在要退出了”。

2. 生命周期

   a. **构造阶段**

      i. **时机**：`new Scene_Gameplay()` 被调用时。

      ii. **行为**：仅分配场景类本身的内存，**不进行**任何重资源（纹理/模型）加载或 ECS 初始化。保持极低开销。

   b. **加载阶段**

      i. **时机**：场景切换发生后，主循环的第一帧。

      ii. **行为**：

         1. **资源预热**：加载 JSON 配置文件、纹理、音频。
         2. **世界构建**：初始化物理世界（Jolt Physics System）、导航网格（NavMesh）。
         3. **实体生成**：解析关卡 JSON，通过 PrefabFactory 批量生成初始实体（Registry 填充）。
         4. **系统启动**：调用所有 System 的 `OnAwake()`。

   c. **活跃循环阶段**

      i. **时机**：游戏运行的每一帧 (Update/FixedUpdate)。

      ii. **行为**：

         1. **输入处理**：处理玩家操作。
         2. **物理步进**：`Sys_Physics.FixedUpdate()`。

3. **逻辑运算**：`Sys_Gameplay.Update()`。
4. **渲染提交**：`Sys_Render.Update()`。
5. **切换检测**：检查 `NextScene` 变量，若非空，准备进入下一阶段。

d. **卸载阶段**

i. **时机**：检测到切换请求后的**帧末**

ii. **行为**：

1. **系统停机**：调用所有 System 的 `OnDestroy()`，停止音频播放。
2. **实体清理**：调用 `Registry.Clear()`，回收所有活动实体 ID。
3. **资源卸载**：释放本场景独占的纹理和内存资源。

e. **销毁阶段**

i. **时机**：`delete CurrentScene` 被 SceneManager 调用时。

ii. **行为**：彻底释放场景对象内存，指针置空。

3. 场景的切换机制

a. **场景状态机**

- **单例管理**：游戏必须维护一个全局的 `SceneManager`，用于持有 `CurrentScene` 指针。
- **延迟切换机制**：
  - **禁止**在 `OnUpdate` 中直接 `delete CurrentScene`。这会导致正在执行的代码访问已释放内存（Use-After-Free）。
  - **约定**：切换请求必须写入 `NextScene` 变量。`SceneManager` 必须在**帧末**检查该变量。若非空，则执行 `CurrentScene->OnExit()` -> `delete CurrentScene` -> `CurrentScene = NextScene` -> `CurrentScene->OnEnter()` 的安全序列。

b. **数据驱动的场景加载**

- **JSON 定义**：场景应当是泛型的（如 `Scene_Gameplay`），其具体内容（敌人位置、地形结构）由 JSON 文件定义。
- **加载流程**：
  1. `OnEnter()` 读取指定的 `Level_X.json`。
  2. 解析 JSON 中的 `Entities` 数组。
  3. 调用 `PrefabFactory` 实例化所有对象到 Registry。
  4. 调用 `Sys_Physics::BuildWorld()` 根据实体碰撞体构建物理空间。

c. **资源保留与释放策略**

- **全局资源 (Global Context)**：跨场景共享的数据（如 `Registry` 的内存池容量、`InputSystem` 的手柄状态、`NetworkClient` 的连接）**不应**随场景销毁。它们属于 `Engine` 层级。
- **场景资源 (Local Context)**：关卡特定的数据（如当前关卡的导航网格 NavMesh、关卡背景音乐 BGM）**必须**在 `OnExit()` 中彻底释放，防止内存泄漏。

d. **场景类型定义**

- 建议至少实现以下基础场景类，以支持完整的游戏流程：
  - `Scene_Menu`：仅包含 UI 系统，不加载物理和 ECS 逻辑。
  - `Scene_Loading`：用于在后台异步加载下一个大场景时的过渡。
  - `Scene_Gameplay`：核心游戏场景，包含完整的 ECS 系统栈。
  - `Scene_NetworkLobby`：多人模式大厅，仅包含网络同步和 UI 系统。

3.3.3 实体 (Entity) 的创建、销毁与标识约定（Entity Creation，Destroy&marking）

1. 简介：（Info）

   实体的用途其实很简单——身份标识，具体的大家应该都知道了，我也就不多赘述了

2. 生命周期

   a. **申请**

      i. **调用**：`Factory` 调用 `registry.create()`。

      ii. **行为**：Registry 从回收池或新计数器中分配一个未使用的 ID（例如 `Index: 10, Version: 1`）。

      iii. **状态**：此时实体是“空”的，没有任何组件。

   b. **组装**

      i. **调用**：`registry.emplace<C_Transform>(entity, ...)`。

      ii. **行为**：在各个组件池中为该 ID 分配空间并写入初始数据。通常由 **Prefab (预制体)** 定义来驱动这一步。

      iii. **标记**：根据用途打上标签组件（Tag Component），如 `C_Tag_Player`。

   c. **活跃**

      i. **行为**：System 在遍历时扫描到该 ID 拥有它关注的组件，开始处理逻辑。

      ii. **变化**：组件数据被不断读写（位置改变、血量减少）。

d. **标记销毁**

   i. **时机**：比如 HP <= 0。

   ii. **行为**：System 调用 `registry.destroy(entity)`。

   iii. **注意**：此时 ID **不会**立即失效，只是被打上了“待删除”标记。在当前帧的后续 System 中，它依然是有效的（为了保证一帧内逻辑的完整性）。

e. **回收**

   i. **时机**：**帧末** 或 **场景卸载**。

   ii. **行为**：Registry 真正移除所有关联组件的数据，并将该 ID 的 **Version + 1**，放入空闲队列。

   iii. *例子*：`Index: 10, Version: 1` 变成了 `Index: 10, Version: 2`。防止旧的引用错误地访问到新实体。

3. 约定

a. **实体本质约定**

- **ID 类型**：实体必须通过 `entt::entity`（或自定义的 `uint32_t`）进行传递。严禁使用裸指针（Raw Pointer）或引用来持久化存储实体。
- **无逻辑性**：实体本身不得包含任何成员函数。所有逻辑必须在 System 中实现。

b. **创建与工厂模式**

- **禁止裸创建**：严禁在 System 业务逻辑中直接调用 `registry.create()` 并手动挂载组件。
- **统一入口**：所有实体的创建必须通过 `PrefabFactory` 类的静态方法（如 `CreateEnemyOrc()`）或数据驱动的 `SpawnPrefab(PrefabID)` 接口进行。这是为了确保组件组装的完整性和初始化的一致性。

c. **标识与检索 (Tags & Identification)**

- **Tag 组件**：对于单例实体（如 Player）或特定类别（如 Enemy），必须挂载空的 Tag 组件（如 `C_Tag_Player`）进行标识。
- **禁止硬编码 ID**：严禁在代码中假定实体的 ID 为固定值（如 `if (id == 0)`）。必须通过 Tag 筛选或 `SceneContext` 获取关键实体。
- **Debug 名称**：所有实体在创建时**必须**挂载 `C_DebugName` 组件，存储可读的字符串名称（如 "Orc_Grunt_01"），以便于 RenderDoc 调试和日志追踪。

d. **延迟销毁机制 (Deferred Destruction)**

- **安全删除**：System 中检测到实体死亡时，应调用 `registry.destroy(entity)`。

- **帧末清理**：Registry 必须配置在每帧的主循环结束时（`OnUpdate` 之后，`SwapBuffers` 之前）执行内存回收。这确保了如果一个实体在 `Sys_Physics` 中死亡，后续的 `Sys_Render` 依然能在本帧正确渲染其“死亡瞬间”，而不是导致访问崩溃。

3.3.4 组件 (Component) 的内存布局与数据约束（Component RAM layout & Data Constraints）

1. 简介（Info）

   正如上面3.2.3中所说的一样，组件的核心用途只有一个，那就是存储纯数据，不包含任何业务逻辑，没有析构函数，内存必须是连续的

2. 内存（Memory）

   a. **挂载 (Attachment)**

      - **时机**：Entity 创建时（Factory）或运行时动态添加（Buff）。
      - **行为**：Registry 在对应的组件池中分配空间，调用**放置构造 (Placement New)**，直接在内存里初始化数据。
      - **状态**：数据就位，System 可以查询到了。

   b. **读写 (Access)**

      - **时机**：System 的 `Update()` 循环中。
      - **行为**：System 通过 `registry.view<T>()` 拿到组件的引用（`T&`）。
      - **特点**：**高频读写**。一帧内可能被读取 10 次，写入 2 次。因此必须保证访问速度极快（O(1)）。

   c. **移除 (Detachment)**

      - **时机**：Entity 销毁时，或逻辑决定移除（比如 Buff 过期）。
      - **行为**：Registry 将该组件的数据与组件池中**最后一个元素交换 (Swap)**，然后**弹出 (Pop)** 最后一个元素。
      - **注意**：这样做是为了保持数组紧凑，不留任何空值。

   d. **序列化 (Serialization)**

      - **时机**：保存游戏进度，或网络同步。
      - **行为**：直接把内存块 `memcpy` 出来，或者转换成 JSON 字符串。因为是 POD，所以不需要复杂的序列化代码。

3. 约定

   a. **POD (Plain Old Data) 强制约束（POD Mandatory Restraint）**

- **纯数据结构体**：所有 Component 必须定义为 `struct`，且仅包含基础数据类型（`float`，`int`，`bool`，`glm::vec3` 等）。
- **禁止复杂类型**：
- **No Complex Types**
  - **严禁**使用 `std::vector`、`std::string`、`std::map` 等动态分配内存的容器。字符串必须使用固定长度数组（如 `char name[32]`）或哈希值（`uint32_t name_hash`）。
  - **严禁**定义虚函数（`virtual`），这会引入虚表指针（vptr），破坏内存布局并增加调用开销。
  - **严禁**在组件中持有裸指针（Raw Pointer）指向其他组件或系统。引用关系必须通过 **Entity ID** 进行。

b. **内存布局与对齐 (Layout & Alignment)**

- **数据紧凑性**：组件成员变量应按照内存对齐原则（Alignment）排列，以减少内存填充（Padding）浪费。建议将大的变量（`vec3`，`float`）放在前面，小的变量（`bool`，`byte`）放在后面。
- **缓存友好 (Cache Friendly)**：为了最大化 CPU 缓存命中率，组件的大小（Size）应尽量控制在 **64 字节 (Cache Line Size)** 以内。若数据过大，应拆分为多个小组件（如 `C_BigData` -> `C_DataA`，`C_DataB`）。

c. **组件分类约定(Component Classification Convention)**

- **数据组件 (Data Component)**：包含业务数据的结构体，如 `C_Health { float hp; }`。这是逻辑运算的主体。
- **标签组件 (Tag Component)**：**空结构体 (Empty Struct)**，不包含任何成员变量。仅用于标记实体身份，如 `struct C_Tag_Player {};`。Registry 对标签组件有专门的优化，不占用每实体的内存空间。
- **共享组件 (Shared Component)**：对于大量实体共用的数据（如 `Mesh` 资源指针、`Material` 参数），应使用 **享元模式 (Flyweight Pattern)**，在组件中仅存储资源 ID 或句柄（Handle），而非资源本身。

d. **序列化与网络同步支持（serialization & Network Synchronization support）**

- **反射支持 (Reflection)**：所有组件结构体必须配合宏（如 `REFLECT_STRUCT`）注册元数据，以支持自动化的 JSON 序列化和编辑器检视（Inspector）。
- **确定性 (Determinism)**：组件中的浮点数运算应保持确定性，避免在网络同步中因精度问题导致的状态发散。

3.3.5 系统 (System) 的无状态性与执行逻辑约定(System:statelessness & Execution Convention)

1. 简介（Info）

   系统在之前的3.2.4也简单介绍了一下，大家其实最需要注意的，就是不要把系统写成“一个超级大类”，适当拆分逻辑，不然出问题自己都不知道怎么出的

   系统最重要的一点其实就是**无状态**——这意味着你可以随时创建、销毁甚至复制一个 System，而不会影响游戏逻辑。逻辑的状态完全由 Component 里的数据决定，但有一个例外，就是System其实是可以持有缓存（Cache）或者配置（Config）的，但这属于环境上下文，不属于游戏的状态

2. 生命周期(lifeCycle)

   a. **构造/唤醒 (Construction / OnAwake)**

      - **时机**：场景加载（`Scene::OnEnter`）时。
      - **行为**：
        - **注册**：向 Registry 注册自己关心的组件类型（`Registry.Group<Pos, Vel>()`）。
        - **初始化**：分配必要的内部资源（如物理引擎初始化、渲染管线创建）。
        - **订阅**：向 EventBus 订阅感兴趣的事件（如 `Subscribe<Event_Collision>()`）。

   b. **执行循环 (Execution Loop / OnUpdate)**

      - **时机**：每帧的主循环中。
      - **行为**：
        - **查询**：`view = registry.view<CompA, CompB>()`。
        - **迭代**：`view.each([](auto entity, auto& a, auto& b) { ... })`。
        - **逻辑**：根据 `a` 和 `b` 的数据计算新状态，或者生成事件。
      - **分阶段**：
        - **FixedUpdate**：物理、AI、网络同步（固定时间步，如 60Hz）。
        - **Update**：输入响应、动画状态更新、UI 刷新（变动时间步，如渲染帧率）。
        - **LateUpdate/Render**：相机跟随、渲染提交（确保逻辑已完成）。

   c. **销毁/清理 (Destruction / OnDestroy)**

      - **时机**：场景卸载（`Scene::OnExit`）时。
      - **行为**：
        - **释放资源**：关闭物理引擎，释放渲染纹理。
        - **取消订阅**：从 EventBus 中移除自己的回调函数，防止野指针调用。

3. 约定(Convention)

a. **无状态原则 (Stateless Principle)**

- **纯逻辑单元**：System 类应当设计为**无状态的函数集合 (Stateless Functors)**。严禁在 System 成员变量中存储**任何游戏运行时状态**（如 `m_player_hp`，`m_enemy_count`）。
- **数据来源**：所有状态数据必须且只能通过 `Registry` 获取组件（Component）来访问，或通过 `SceneContext` 获取全局单例资源（如 `Res_Time`，`Res_Input`）。
- **例外**：System 允许持有**外部系统的句柄 (Handles)** 或 **只读配置数据 (Read-only Config)**，例如：
  - `PhysicsSystem` 持有 `Jolt::PhysicsSystem*`。
  - `RenderSystem` 持有 `NCL::Renderer*`。
  - `AISystem` 持有行为树模板数据。

b. **核心执行阶段 (Execution Phases)**
为了确保因果链的正确性（如物理->逻辑->渲染），所有 System 必须实现并严格遵循以下生命周期接口：

- `OnAwake(Registry&)`：
  - 系统初始化。仅在此阶段分配内存、加载配置、注册 EventBus 监听器。
- `OnFixedUpdate(Registry&, float fixedDt)`：
  - **物理与核心逻辑**。
  - 包含：物理模拟 (`Sys_Physics`)、AI 决策 (`Sys_AI`)、状态机更新 (`Sys_FSM`)、网络同步 (`Sys_Network`)。
  - **强制约束**：此阶段必须使用固定时间步长（如 0.016s），确保跨平台确定性。
- `OnUpdate(Registry&, float dt)`：
  - **表现与输入**。
  - 包含：输入处理 (`Sys_Input`)、动画状态计算 (`Sys_Animation`)、UI 刷新 (`Sys_UI`)、音效触发 (`Sys_Audio`)。
  - **强制约束**：此阶段使用变长时间步长，逻辑必须能够处理帧率波动。
- `OnDestroy(Registry&)`：
  - 系统卸载。必须在此阶段释放所有非托管资源（如 GPU 句柄、物理世界内存），并取消所有 EventBus 订阅。

c. **依赖管理与执行顺序(Dependencies Management & Execution Sequences)**

- **显式依赖**：System 之间不得直接调用对方的函数（如 `Sys_Physics.DoCollision()`）。交互必须通过 **EventBus** 或 **修改组件数据** 间接进行。

- **执行顺序 (System Ordering)**：`SceneManager` 需维护一个有序的 System 列表（`SystemGroup`）。
  - 示例顺序：Input -> Network -> Physics -> Gameplay -> Animation -> Audio -> Render。
  - Render System 必须始终位于列表末尾，确保渲染的是当前帧逻辑计算后的**最终状态**。

3.3.6 事件 (Event) 通信机制与使用场景约定(Event Communication & Sence Usage)

1. 简介(Info)

   事件我们也简单的在3.2.5中解释了，它的主要作用就是解耦

   打个比方，`Sys_Physics` 不需要知道 `Sys_Gameplay` 的存在，也不需要引用它的头文件。而一个“死亡事件”，可能同时被 UI（显示击杀图标）、音频（播放惨叫）、任务（击杀计数+1）和掉落（生成金币）四个系统监听。如果用函数调用写，代码会乱成一团麻。

   因此，只要我们控制好分发（Dispatch）事件的时机，我们就可以精准的控制逻辑执行的顺序

2. 生命周期

   事件的生命周期极短，通常只有 **几毫秒**（帧内）或 **一帧**。

   a. **发布 (Publish)**
      - **动作**：`Sys_Physics` 创建 `Event_Collision` 结构体。
      - **行为**：调用 `bus.publish(event)`。
      - **状态**：事件被推入 EventBus 内部的 `std::vector` 队列中。**此时没有任何回调函数被执行**。

   b. **缓冲 (Buffering)**
      - **状态**：事件静静地躺在内存里。
      - **意义**：允许系统在不打断自身执行流的情况下，连续产生大量事件（如爆炸产生的 100 个碎片）。

   c. **分发 (Dispatch / Flush)**
      - **动作**：主循环 (Main Loop) 显式调用 `bus.flush<EventType>()`。
      - **行为**：EventBus 遍历该类型的队列，依次调用所有订阅者（Subscriber）的 `OnEvent(const Event&)` 函数。
      - **关键**：这是**逻辑真正执行**的时刻。

   d. **处理 (Handling)**
      - **动作**：订阅者（如 `Sys_Gameplay`）接收数据，修改 Component 状态（如扣血）。

- **连锁反应**：处理过程中可能会产生新的事件（如 `Event_Death`），新事件会进入下一轮缓冲。

e. **清理 (Clear)**

- **动作**：帧末 (Frame End) 调用 `bus.clear()`。
- **行为**：清空所有队列的 `size`（保留 `capacity` 以避免内存重新分配）。
- **状态**：一切归零，准备下一帧。

3. 约定(Conventions)

a. **混合式事件总线架构 (Hybrid Event Bus)**
本项目采用 **“发布-订阅” (Pub-Sub)** 模式的事件总线，但在分发策略上支持两种模式，以平衡性能与即时性：

- **立即分发模式 (Immediate Phased Dispatch)**
  - **定义**：事件在生成后，必须在**当前帧的特定逻辑阶段**被立即处理。
  - **场景**：`Event_Collision`（碰撞）、`Event_TakeDamage`（伤害）、`Event_Death`（死亡）。
  - **目的**：确保物理因果链（碰撞->伤害->死亡->掉落）在同一帧内闭环，避免视觉上的“延迟感”或逻辑上的“穿模”。
- **延迟分发模式 (Buffered Dispatch)**
  - **定义**：事件被推入队列，统一在**帧末 (End of Frame)** 进行处理。
  - **场景**：`Event_PlaySound`（音效）、`Event_ShowUI`（飘字）、`Event_UnlockAchievement`（成就）。
  - **目的**：最大化解耦，避免非关键逻辑阻塞主循环的核心计算。

b. **核心分发时序约定**
主循环 (`GameLoop`) 必须严格遵守以下事件刷新（Flush）顺序，严禁随意更改：

i. **Sys_Physics.FixedUpdate()** 执行完毕。

ii. **Flush(Event_Collision)**：立即处理碰撞，转化为伤害/位移逻辑。

iii. **Sys_Gameplay.FixedUpdate()** 执行完毕。

iv. **Flush(Event_Death/Quest)**：立即处理死亡，生成掉落物，更新任务状态。

v. **Sys_Render.Update()**：基于最新状态进行渲染插值。

vi. **Frame End**：处理剩余的延迟事件（音效/UI），并调用 `Bus.Clear()`。

vii. **事件数据结构约束**

- **POD 类型**：所有事件必须定义为 `struct`，仅包含纯数据。
- **禁止逻辑**：事件结构体中严禁包含任何逻辑函数（如 `event.DoDamage()`）。

- **引用安全**：
  - 推荐在事件中传递 **Entity ID** 而非指针。
  - 若必须传递指针（如 `CollisionData*`），必须确保该指针在分发阶段依然有效，严禁传递栈上临时对象的地址。

c. **订阅者规范（Subscribers Regulations）**

- **只读访问**：订阅者接收到的事件应为 `const Event&`，禁止修改事件本身的数据来传递返回值（EventBus 是单向通知，不是双向请求）。
- **轻量处理**：`OnEvent` 回调函数中不应执行耗时操作（如 I/O 读取、路径搜索）。若需耗时操作，应仅在回调中设置标记（Dirty Flag），在 System 的 `Update` 阶段异步处理。

3.3.7 全局资源 (Global Resource) 的挂载与访问约定(Global Resource Attachments & Acess Conventions)

1. 简介(Info)

ECS架构有一个很不“编程友好”的问题——如果所有的数据全都依附于 `Entity` 的话，那全局变量，比如时间，输入，配置，游戏全局状态该怎么办?

有人就会说：那我们创建一个 `GlobalEntity`，然后每次有需要就去查找它就好了啊

但这太麻烦，太浪费，也太不直观了

因此我们需要退一步，进两步，创建一个公共的变量仓库来管理全局变量

2. 生命周期

全局资源的生命周期通常与 **`Registry`** 或 **`Scene`** 一致。

a. **挂载 (Mount / Create)**

- **时机**：场景加载 (`Scene::OnEnter`) 或 游戏启动 (`App::Init`)。
- **行为**：
  - `registry.set<Res_Time>()`：在 Registry 内部创建一个 `Res_Time` 实例。
  - 初始化默认值（`timeScale = 1.0f`）。

b. **更新 (Update / Modify)**

- **时机**：System 的 `Update` 循环中。
- **行为**：
  - `Sys_Input` 每帧都会修改 `Res_Input` 的数据（写入鼠标坐标）。
  - `Sys_Gameplay` 每帧都会读取 `Res_Input` 来控制玩家移动。
  - `Sys_Physics` 每帧读取 `Res_Time.fixedDt` 来进行模拟。

c. **访问 (Access)**
   - **行为**：任意 System 都可以通过 `registry.ctx<T>()` 获取资源的引用。
   - **特点**：极快（O(1)），通常是指针跳转。
d. **卸载 (Unmount / Destroy)**
   - **时机**：场景卸载 (`Scene::OnExit`)。
   - **行为**：
     - `registry.unset<Res_Time>()`：销毁资源实例。
     - 或者随 Registry 一起销毁。

3. 约定(Conventions)
   a. **上下文单例 (Context Singleton)**
      - **定义**：全局资源是指**不依赖于特定 Entity**、在整个 ECS 世界中**唯一存在**的数据结构体。
      - **存储位置**：所有全局资源必须存储在 `Registry` 的**上下文 (Context)** 容器中，严禁使用全局静态变量（`static`）或单例模式（`Singleton::Instance()`）来绕过 ECS 架构。
      - **访问方式**：System 必须通过 `registry.ctx<T>()` 或 `registry.try_get<T>()` 接口访问资源。
   b. **核心资源类型约定**
      为了规范开发，本项目预定义以下核心全局资源，严禁随意增删：
      - `Res_Time` **(时间)**：
        - 包含：`deltaTime` (上一帧耗时), `fixedDeltaTime` (物理步长), `timeScale` (时间流速), `totalTime` (游戏运行总时长)。
        - *维护者*：`GameLoop` (主循环)。
      - `Res_Input` **(输入)**：
        - 包含：`mousePos` (鼠标位置), `keyStates` (键盘按键位图), `axisValues` (手柄摇杆值)。
        - *维护者*：`Sys_Input`。
      - `Res_GlobalConfig` **(配置)**：
        - 包含：`gravity` (重力向量), `screenSize` (分辨率), `volume` (音量设置)。
        - *维护者*：`SceneLoader` (在加载时读取 JSON 填充)。
      - `Res_GameState` **(游戏状态)**：
        - 包含：`score` (得分), `enemyCount` (当前敌人数量), `isPaused` (暂停标记)。
        - *维护者*：`Sys_Gameplay`。

c. **读写权限约定(IO Permission Rules)**

- **单一写原则 (Single Writer Principle)**：
  - 为了防止数据竞争（Data Race）和逻辑混乱，建议每个全局资源**仅由一个特定的 System 负责写入**（如 `Sys_Input` 写 `Res_Input`）。
  - 其他所有 System **只读**该资源。
- **例外**：`Res_GameState` 可能需要被多个逻辑系统修改（如 `Sys_Combat` 加分，`Sys_Quest` 改任务状态），此时需注意逻辑顺序或使用原子操作。

i. **生命周期管理**

- **随场景重置**：大部分全局资源（如 `Res_GameState`，`Res_Input`）应在 `Scene::OnExit()` 时被清除或重置，防止上一关的状态污染下一关。
- **跨场景持久化**：若需跨场景保留数据（如 `Res_PlayerProfile` 玩家存档），应将其存储在 `GameInstance` 或 `Application` 层级，并在新场景加载时重新注入 Registry。

3.3.8 预制体 (Prefab) 组装与实例化流程约定（Prefab Assembly & Instantiation Conventions）

1. 简介（Info）

   prefab的话，大家应该在unity或者ue里面见到过了

   简而言之，prefab是一种指导Registry如何生产Entity的**说明书**

   它是一个包含实体和该实体所拥有的组件的json文件，Registry会根据这个说明书，生产出对应的实体和它所带有的组件

2. 生命周期

   预制体的生命周期分为 **定义阶段** 和 **实例化阶段**。

   a. **定义 (Definition / JSON Authoring)**

   - **时机**：开发阶段，手动编写或编辑器生成。
   - **行为**：创建一个 JSON 文件，如 `Assets/Prefabs/Prefab_Enemy_Orc.json`。
   - **内容**：

代码块

```
{
  "PrefabID": "PREFAB_ENEMY_ORC",
  "Name": "Orc_Grunt",
  "Components": {
      "C_D_Transform": { "pos": [0,0,0], "rot": [0,0,0,1] },
      "C_D_Health": { "current": 100, "maximum": 100 },
      "C_D_MeshRenderer": { "meshHandle": "orc.obj", "textureHandle": "orc.png" },
```

```
        "C_D_AI": { "behaviorTree": "BT_Patrol" },
        "C_T_Enemy": {}
    }
  }
```

b. **加载 (Load / Parsing)**
   - **时机**：游戏启动或场景加载时。
   - **行为**：`PrefabManager` 读取所有 JSON 文件，解析成内存中的 `PrefabTemplate` 结构体（比如 `std::map<string, ComponentData>`）。
   - **优化**：避免每次生成实体都读硬盘，而是读内存里的模板。

c. **实例化 (Instantiation / Spawning)**
   - **时机**：`Sys_Spawner` 或 游戏逻辑需要生成敌人时。
   - **行为**：
     1. 调用 `PrefabFactory::Spawn("Orc_Grunt", position)`。
     2. `Registry.create()` 分配 ID。
     3. 遍历模板中的组件列表。
     4. 对每个组件调用 `Registry.emplace<T>(id, data)`，把模板数据**拷贝**进去。
     5. 覆盖位置信息（用传入的 `position` 覆盖模板里的默认 `[0,0,0]`）。

d. **销毁 (Destruction)**
   - **行为**：与普通 Entity 一样，被 `Registry.destroy(id)` 回收。

3. 约定（Conventions）

a. **预制体定义规范（Prefab Defination Regulation）**
   - **JSON 格式**：所有预制体必须以 JSON 格式存储在 `Assets/Prefabs/` 目录下。文件名必须与 Prefab ID 一致（如 `Prefab_Enemy_Orc.json` 对应 ID `"PREFAB_ENEMY_ORC"`）。
   - **组件映射**：JSON 中的键名（Key）必须与 C++ 组件结构体的反射名称（Reflection Name）严格对应（如 `"Transform"` 对应 `C_Transform`）。
   - **默认值**：若 JSON 中省略某组件字段，工厂必须使用该组件结构体的 C++ 默认构造函数值。

b. **工厂模式实现 (Prefab Factory)**
   - **静态工厂类**：实现一个全局静态类 `PrefabFactory`，提供统一的实例化接口：
     - `EntityID Spawn(Registry&, const string& prefabID, vec3 position, quat rotation)`

- **模板缓存**：为了保证性能稳定，`PrefabFactory` 必须在 `OnAwake()` 阶段预加载所有 JSON 文件，将其解析为内存中的 `PrefabTemplate` 对象。严禁在 `Spawn()` 过程中进行文件 I/O 操作。

c. **组件挂载流程**
实例化一个预制体时，必须严格遵循以下步骤：

i. **创建实体**：向 Registry 申请一个新的 Entity ID。

ii. **挂载必选组件**：自动挂载 `C_Transform` (位置)、`C_Tag` (类型标签)、`C_DebugName` (调试名)。

iii. **挂载数据组件**：遍历 JSON 定义，反射生成对应的组件（如 `C_Health`，`C_Mesh`）并填充数据。

iv. **覆盖初始状态**：使用 `Spawn()` 传入的参数（位置、旋转）覆盖 `C_Transform` 的默认值。

v. **触发回调**：若组件需要后续初始化（如 `C_Physics` 需要创建 Jolt Body），必须在所有组件挂载完成后，发送一个 `Event_EntitySpawned` 事件，由各系统自行处理。

d. **变体与继承 (Variants & Inheritance)**

- **原型继承**：支持 `Parent` 字段。如 `Elite_Orc.json` 可以定义 `"Parent": "Enemy_Orc"`，仅需覆盖差异部分（如 `"Health": 200"`），减少数据冗余。
- **运行时修改**：允许在 Spawn 后立即通过代码修改组件数据（如 `entity.get<C_Health>().hp = 50`），实现动态生成的变体。
- **Prototype Inheritance**: Supports `Parent` fields like: `Elite_Orc.json` can define `"Parent": "Enemy_Orc"`, requiring only the differing parts to be overridden (e.g., `"Health": 200`), reduce data redundancy.
- **Runtime Modification**: Allows component data to be modified immediately after spawning via code (e.g., `entity.get<C_Health>().hp = 50`), enabling dynamically generated variants.

3.3.9 资源句柄 (Asset Handle) 与引用规范(Asset Handle & Reference Regulations)

1. 简介(Info)

资源句柄的本质就是能够**安全的，简单的**引用资源，就比如说，假如我们有一百个敌人，总不可能每个敌人都存一张贴图对象吧，我们只需要**统一管理**，存一个贴图id即可——这样也能够把指针解耦了，避免显存满了，纹理一释放，指针就没有用了

2. 生命周期

a. **加载 (Load / Request)**

- **调用**：`Sys_Render` 在初始化或 Prefab 实例化时，请求资源。

- **行为**：
  1. `AssetManager::Get<Texture>("orc.png")`。
  2. 检查哈希表（Cache）。
  3. **命中**：直接返回已有的 `HandleID`，引用计数 +1。
  4. **未命中**：从硬盘读取 `orc.png` -> 上传显存 -> 生成新 `HandleID` -> 引用计数 = 1 -> 返回 ID。

b. **持有 (Hold / Store)**
- **位置**：Component 中（如 `C_Mesh` 存 `MeshHandle`，`C_Material` 存 `TextureHandle`）。
- **形式**：通常是一个 `uint32_t` 或 `AssetHandle` 结构体。

c. **使用 (Use / Access)**
- **调用**：`Sys_Render` 在渲染循环中。
- **行为**：
  1. 从 `C_Mesh` 拿出 `MeshHandle`。
  2. 调用 `AssetManager::Resolve(handle)` 获取真正的 OpenGL 资源 ID。
  3. 提交给 GPU。

d. **释放 (Release / Unload)**
- **时机**：Entity 销毁，或 Scene 卸载。
- **行为**：
  1. Component 被析构（或重置）。
  2. 调用 `AssetManager::Release(handle)`，引用计数 -1。
  3. **检测**：若引用计数归零，且资源策略允许（如 `UnloadUnusedAssets`），则真正释放显存/内存。

3. 引用规范

a. **句柄类型定义 (Handle Types)**
- **强类型 ID**：所有资源引用必须使用强类型句柄（Strongly Typed Handle），严禁直接在组件中存储裸指针（Raw Pointer）或资源路径字符串。
  - `using TextureHandle = uint32_t;`
  - `using MeshHandle = uint32_t;`
  - `using AudioHandle = uint32_t;`
- **无效句柄**：必须定义 `INVALID_HANDLE = 0`，用于表示空资源或加载失败状态。

b. **资源管理器职责 (AssetManager Responsibility)**

- **单例管理**：实现全局单例 `AssetManager`，负责所有外部资产（Texture, Mesh, Shader, Audio）的生命周期。
- **缓存机制**：
  - 内部维护 `std::unordered_map<string, ResourceData>`。
  - 提供 `Load<T>(path)` 接口：若资源已存在，直接返回句柄并增加引用计数；若不存在，执行加载流程。
- **引用计数**：所有资源必须维护引用计数（Ref Count）。当引用计数归零时，资源不应立即卸载，而是进入“待回收队列”，仅在 `Scene::OnExit()` 或显式调用 `UnloadUnused()` 时释放，以避免频繁 IO 抖动。

c. **组件引用规范**

- **数据组件**：
  - `C_MeshRenderer`：包含 `MeshHandle mesh` 和 `MaterialHandle material`。
  - `C_AudioSource`：包含 `AudioHandle clip`。
- **禁止深拷贝**：组件拷贝时仅拷贝句柄 ID（整数），严禁触发资源的深拷贝（Deep Copy）。
- **资源预热**：为了避免游戏过程中的卡顿（Hitch），建议在 `Scene::OnEnter()` 阶段根据 JSON 配置列表，批量预加载（Preload）所有可能用到的资源，预先获取句柄。

d. **默认资源回退 (Fallback)**

- **紫黑格纹理**：若请求的纹理加载失败（文件丢失），AssetManager 必须返回一个内置的“紫黑格”错误纹理句柄，确保游戏不会崩溃且能明显提示资源缺失。
- **默认网格**：若请求的模型加载失败，必须返回一个内置的“立方体”或“球体”网格句柄。

3.4 职责定义与索引一览表

3.4.1 [表] 核心实体 (Entity) 定义一览表

表格

	实体名称	代码变量命名示例	描述	必备核
1	Entity_Player	entity_Player_Main	玩家主控实体	C_T_
2	Entity_Enemy_Orc_Grunt	entity_Enemy_Orc_01	基础近战敌人	C_T_
3	Entity_Enemy_Orc_Elite	entity_Enemy_Orc_Elit...	强化敌人（高警戒）	C_T_
4	Entity_Item_Key	entity_Item_Key_Blue...	任务道具实体	C_T_
5	Entity_Projectile_Dart	entity_Projectile_Dart_...	飞行道具/投射体	C_T_
6	Entity_Trigger_Interact	entity_Trigger_Door_01	场景交互触发器	C_T_

6 条记录

3.4.2 [表] 核心标签 (Tag) 定义一览表

表格

	标签名称 (Tag Name)	逻辑含义	典型持有者	用途示例
1	PlayerTag	玩家控制的主体	主角	相机跟随的目标、
2	EnemyTag	敌对单位	兽人、Boss	索敌系统的搜索目
3	InteractableTag	可交互物体	宝箱、NPC	玩家按下E键时的
4	DeadTag	已死亡待清理	尸体	死亡清理系统、无

4 条记录

3.4.3 [表] 渲染层级 (Render Layer) 与 Z轴排序定义

表格

	层级名称 (Layer)	Z轴范围 (Depth)	描述	包含物体示例
1	LAYER_BACKGROUND	-10.0 ~ -1.0	永远在最后面的背景	远景贴图、天空盒
2	LAYER_GROUND	0.0	地面层	地形网格、地板
3	LAYER_OBJECTS	0.1 ~ 5.0	场景物件 (基于Y轴排序)	树木、桌子、石头
4	LAYER_CHARACTERS	0.1 ~ 5.0	动态角色 (基于Y轴排序)	玩家、敌人、NPC
5	LAYER_VFX	6.0 ~ 9.0	特效层	爆炸、魔法光效
6	LAYER_UI	100.0+	用户界面	血条、对话框、HUD

6 条记录

3.4.4 [表] 物理碰撞层 (Collision Layer) 与 矩阵定义

表格

	层级名称	描述	会与哪些层发生碰撞 (M...	备注
1	COLLISION_PLAYER	玩家胶囊体	WALL, ENEMY, TRIGGER	不与子弹碰撞 (防自伤
2	COLLISION_ENEMY	敌人胶囊体	WALL, PLAYER, BULLET	敌人之间可能会互相
3	COLLISION_WALL	静态墙体	ALL	阻挡一切
4	COLLISION_BULLET	飞行道具	WALL, ENEMY	穿透队友 (Player)

4 条记录

3.4.5 [表] 核心组件 (Component) 职责一览表

表格

	组件名称	数据成员 (Data Members)	内存类型	职
1	TransformComponent	vec3 pos, vec3 rot, vec...	POD (纯数据)	世界空
2	HealthComponent	float current, float max	POD	生命值
3	SpriteComponent	AssetHandle textureID, ...	POD (带句柄)	渲染所
4	BuffComponent	std::vector<BuffID> acti...	Dynamic (需注意)	存储当

4 条记录

3.4.6 [表] 核心系统 (System) 职责一览表

表格

	系统名称	职责简述	执行阶段	读
1	MovementSystem	根据输入意图更新位置	FixedUpdate	Input
2	DamageSystem	计算伤害并扣除血量	Update (Post)	Defer
3	RenderSystem	提交精灵到渲染队列	Render	Sprite
4	AISystem	运行行为树并生成意图	Update (Pre)	Perce

4 条记录

3.4.7 [表] 关键事件 (Event) 定义一览表

表格

	事件名称	触发时机	携带数据	监
1	EntityDamageEvent	当攻击判定成功时	TargetID, SourceID, Am...	Dama
2	LevelLoadedEvent	场景加载完成时	LevelName, SpawnPoint	Game
3	ItemPickupEvent	玩家触碰道具时	PlayerID, ItemID	Inven

3 条记录

3.4.8 [表] 全局资源 (Global Resource) 定义一览表

表格

	资源名称	包含数据	更新者	用
1	TimeResource	float dt, float timeScale	引擎主循环	提供
2	InputResource	vec2 mousePos, bool k...	窗口回调函数	供 Inp
3	CameraContext	mat4 view, mat4 proj	CameraSystem	供渲

3 条记录

3.4.9 [表] 标准预制体 (Prefab) 类型一览表

表格

	预制体ID (Enum)	资源路径 / 配置文件	工厂方法名	备
1	PREFAB_ORC_WARRIOR	data/enemies/orc_war...	Factory::CreateOrc()	标准
2	PREFAB_FIREBALL	data/spells/fireball.json	Factory::CreateProjecti...	包含
3				

3 条记录

3.4.10 [表] 输入动作映射 (Input Action Mapping) 一览表

表格

	动作名称 (Action)	触发类型	默认键位 (Keyboard)	默
1	ACTION_MOVE_X	Axis (轴)	A / D	Left S
2	ACTION_ATTACK	Press (按压)	Mouse Left	Butto
3	ACTION_INTERACT	Press (按压)	E	Butto

3 条记录

4. 最终架构依赖图